



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



内容回顾

- **API(Application Programming Interface)**, 应用程序编程接口; **Web API**
- **DOM(Document Object Model)**, 文档对象模型, 处理可扩展标记语言 (**HTML**或者**XML**) 的标准编程接口; **DOM树**
- 获取元素:
 - **document.getElementById('id');**
 - **document.getElementsByTagName('标签名'),**
element.getElementsByTagName('标签名');
 - **document.getElementsByName('name');**
 - **querySelector()、querySelectorAll()和getElementsByClassName()**
- 对象属性: **document.title**



6.4 事件基础

1

事件概述

为元素添加交互行为

事件是指可以被JavaScript侦测到的行为，是一种“**触发-响应**”的机制。这些行为指的就是页面的加载、鼠标单击页面、鼠标指针滑过某个区域等具体的动作，它对实现网页的交互效果起着重要的作用。



6.4 事件基础

2

事件三要素

事件三要素是指事件源、事件类型和事件处理程序这3部分。名词解释如下：

- ❑ **事件源**：触发事件的元素（谁触发了事件）
- ❑ **事件类型**：如 **click** 单击事件（触发了什么事件）
- ❑ **事件处理程序**：事件触发后要执行的代码（函数形式），也称事件处理函数（触发事件以后要做什么）



6.4 事件基础

2

事件三要素

案例演示: 为按钮绑定单击事件，触发按钮事件后，会弹出一个信息。

示例代码

```
<body>
  <button id="btn">单击</button>
  <script>
    var btn = document.getElementById('btn'); // 第1步：获取事件源
    // 第2步：注册事件btn.onclick
    btn.onclick = function () { // 第3步：添加事件处理程序（采取函数赋值形式）
      alert('弹出');
    };
  </script>
</body>
```



6.5 操作元素

1

操作元素内容

在前面的内容中已经讲解了获取元素的几种方式，接下来将利用DOM提供的[属性](#)实现对元素内容的操作。常用的属性如下表所示：

属性	作用
element.innerHTML	设置或返回元素开始和结束标签之间HTML。包括HTML标签，同时保留空格和换行
element.innerText	设置或返回元素的文本内容，在返回的时候会去除HTML标签和多余的空格、换行，在设置的时候会进行特殊字符转义
element.textContent	设置或者返回指定节点的文本内容，同时保留空格和换行



6.5 操作元素

1

操作元素内容

案例演示: 分别利用innerHTML、innerText、textContent属性在控制台输出一段HTML文本。

示例代码

```
<body>
<div id="box">
  The first paragraph...
  <p>
    The second paragraph...
    <a href="http://www.example.com">third</a>
  </p>
</div>
</body>
```

The screenshot shows a browser's developer console with the 'Console' tab selected. It displays the results of three JavaScript commands: 1. `var Obox = document.getElementById('box'); Obox.innerHTML` returns the full HTML content of the box, including the opening and closing tags for the div and body. 2. `Obox.innerText` returns the visible text content, including the text of the paragraph and the link. 3. `Obox.textContent` returns the text content, including the text of the paragraph and the link, but it does not include the opening and closing tags of the div or body.

```
> var Obox = document.getElementById('box')
Obox.innerHTML
< "
  The first paragraph...
  <p>
    The second paragraph...
    <a href="http://www.example.com">third</a>
  </p>
"
```

```
> Obox.innerText
< "The first paragraph...

The second paragraph... third"
```

```
> Obox.textContent
< "
  The first paragraph...

  The second paragraph...
  third
"
```

元素内容



6.5 操作元素

2

操作元素属性

在DOM中，HTML属性操作是指使用JavaScript来操作一个元素的HTML属性。

一个元素包含很多的属性，例如，对于一个img图片元素来说，可以操作它的src、title属性等，或者对于input元素来说，可以操作它的disabled、checked、selected属性等。



6.5 操作元素

2

操作元素属性

例6.5.2-1

案例演示: 以单击按钮操作img元素属性为例进行代码演示。

① 编写HTML结构代码

示例代码

```
<body>
  <button id="flower">鲜花</button>
  <button id="grass">四叶草</button> <br>
  
</body>
```



6.5 操作元素

2

操作元素属性

② 获取元素

示例代码

```
<script>  
  // 1. 获取元素  
  var flower = document.getElementById('flower');  
  var grass = document.getElementById('grass');  
  var img = document.querySelector('img');  
  ...  
</script>
```



6.5 操作元素

2

操作元素属性

③ 注册事件处理程序

示例代码

```
// 2. 注册事件 处理程序
flower.onclick = function () {
    img.src = 'images/flower.png';
    img.title = '鲜花';
};
grass.onclick = function () {
    img.src = 'images/grass.png';
    img.title = '四叶草';
};
```



6.5 操作元素

2

操作元素属性

例6.5.2-2

案例演示: 以单击按钮操作input表单属性为例进行代码演示。

① 编写HTML结构代码

示例代码

```
<body>
  <button>按钮</button>
  <input type="text" value="输入内容">
</body>
```



6.5 操作元素

2

操作元素属性

② 获取元素

示例代码

```
<script>  
  // 1. 获取元素  
  var btn = document.querySelector('button');  
  var input = document.querySelector('input');  
  var img = document.querySelector('img');  
  ...  
</script>
```



6.5 操作元素

2

操作元素属性

③ 注册事件处理程序

示例代码

```
// 2. 注册事件 处理程序
btn.onclick = function () {
    input.value = '被点击了! '; // 通过value来修改表单里面的值
    this.disabled = true;      // this指向的是事件函数的调用者 btn
};
```



6.5 操作元素

3

【案例】显示隐藏密码明文

例6.5.3

案例分析：在登录页面，为了优化用户体验，方便用户进行密码输入。因此在设计密码框时，会有一个“眼睛”图片，充当按钮功能，单击可以切换按钮的状态，控制密码的显示和隐藏。实现步骤如下：

- 准备一个父盒子div
- 在父盒子中放入两个子元素，一个input元素和一个img元素
- 单击眼睛图片切换input的type值（text和password）



6.5 操作元素

3

【案例】显示隐藏密码明文

效果图如下：



隐藏密码明文



显示密码明文



6.5 操作元素

3

【案例】显示隐藏密码明文

代码实现：步骤如下。

① 编写HTML结构，完成页面布局

示例代码

```
<body>
  <div class="box">
    <label for="">
      
    </label>
    <input type="password" name="" id="pwd">
  </div>
</body>
```



6.5 操作元素

3

【案例】显示隐藏密码明文

② 获取元素

示例代码

```
<script>  
  // 1. 获取元素  
  var eye = document.getElementById('eye');  
  var pwd = document.getElementById('pwd');  
</script>
```



6.5 操作元素

3

【案例】显示隐藏密码明文

③ 注册事件处理程序

示例代码

```
// 2. 注册事件 处理程序
var flag = 0;
eye.onclick = function () { // 每次单击, 修改flag的值
    if (flag == 0) {
        pwd.type = 'text';
        eye.src = 'images/open.png';
        flag = 1;
    } else { ... }
};
```



6.5 操作元素

4

操作元素样式

操作元素样式的两种方式：一种是操作style属性，另一种是操作className属性。

① 操作style属性

例6.5.4-1

元素对象的样式，可以直接通过“元素对象.style.样式属性名”的方式操作。

样式属性名对应CSS样式名，但需要去掉CSS样式名里的半字线“-”，并将半字线后面的英文的首字母大写。

帕斯卡命名法

驼峰命名法



6.5 操作元素

4

操作元素样式

常见的style属性操作的样式名如下表：

方法	作用
background	设置或返回元素的背景属性
backgroundColor	设置或返回元素的背景色
display	设置或返回元素的显示类型
fontSize	设置或返回元素的字体大小
height	设置或返回元素的高度
left	设置或返回定位元素的左部位置
listStyleType	设置或返回列表项标记的类型
overflow	设置或返回如何处理呈现在元素框外面的内容
textAlign	设置或返回文本的水平对齐方式
textDecoration	设置或返回文本的修饰
textIndent	设置或返回文本第一行的缩进
transform	向元素应用2D或3D转换



6.5 操作元素

4

操作元素样式

案例演示: 通过代码演示如何操作元素的style属性添加样式。

示例代码

```
<div id="box"> </div>
<script>
  var ele = document.querySelector('#box'); // 获取元素对象
  ele.style.width = '100px';
  ele.style.height = '100px';
  ele.style.transform = 'rotate(7deg)';
  // 上述3行代码相当于在CSS中添加以下样式:
  #box {width: 100px; height: 100px; transform: rotate(7deg);}
</script>
```



6.5 操作元素

4

操作元素样式

② 操作className属性

例6.5.4-1

在开发中，如果样式修改较多，可以采取操作类名的方式更改元素样式，语法为“元素对象.className”。访问className属性的值表示获取元素的类名，为className属性赋值表示更改元素类名。如果元素有多个类名，在className中以空格分隔。



6.5 操作元素

4

操作元素样式

案例演示: 使用className更改元素的样式。

① 编写HTML结构，完成页面布局

示例代码

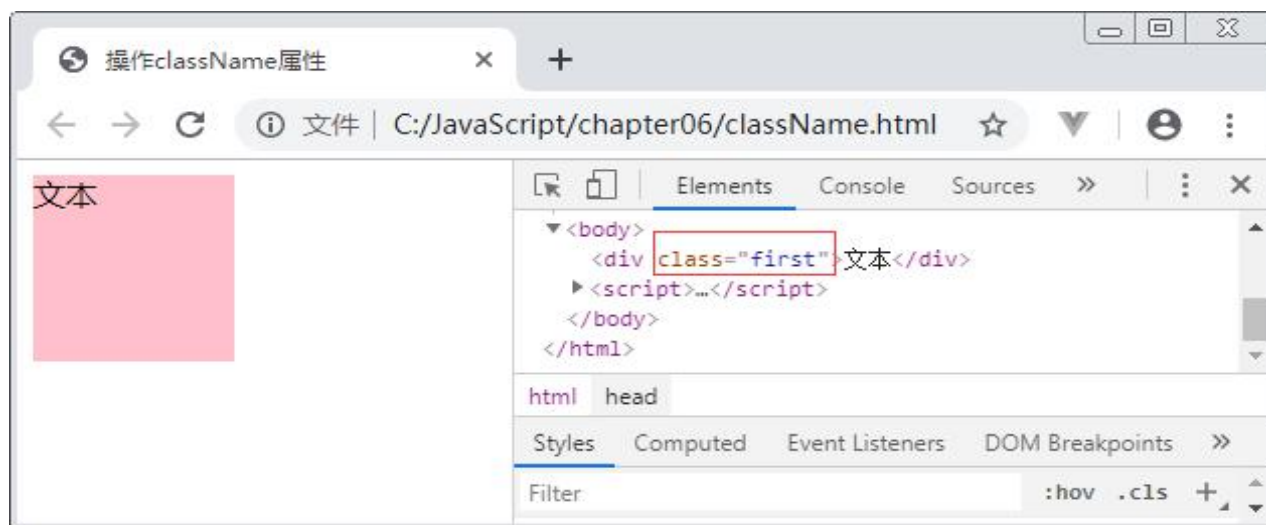
```
<style>
  div { width: 100px; height: 100px;
        background-color: pink;
      }
</style>
<body>
  <div class="first">文本</div>
</body>
```



6.5 操作元素

4

操作元素样式



初始效果图



6.5 操作元素

4

操作元素样式

② 单击div元素更改元素的样式

示例代码

```
<script>
  var test = document.querySelector('div');
  test.onclick = function () {
    this.className = 'change';
  };
</script>
```

② 在style中添加change类

示例代码

```
.change {
  background-color: purple;
  color: #fff;
  font-size: 25px;
  margin-top: 100px;
}
```

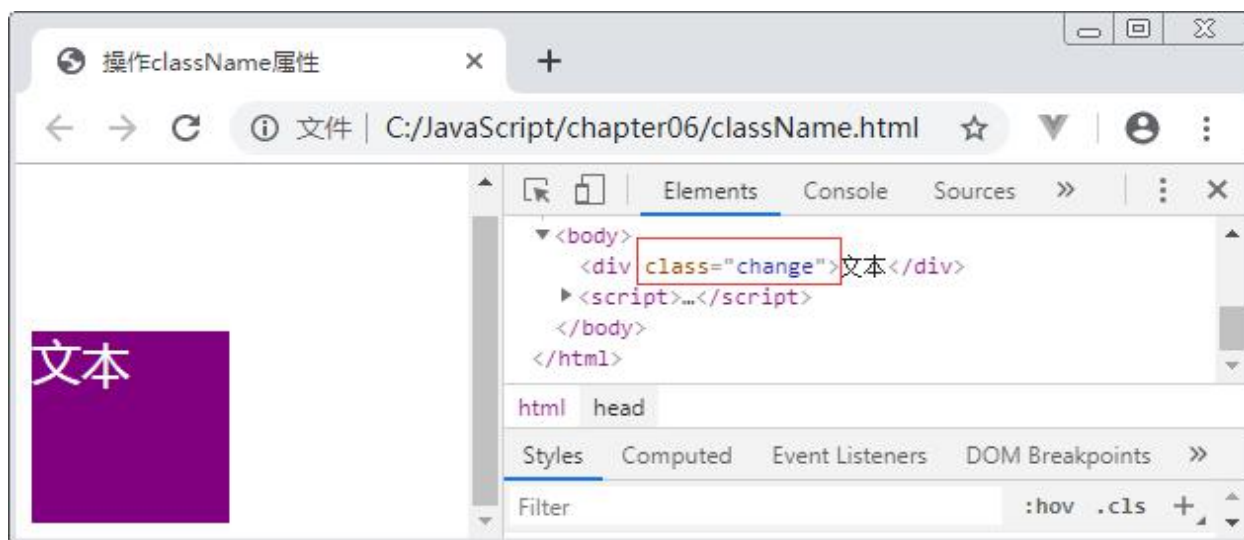
6.5 操作元素

4

操作元素样式

④ 单击

盒子，浏览器预览效果如下



单击后效果图



6.5 操作元素

4

操作元素样式

⑤ 保留原先的类名，修改第（2）步的第5行代码

```
<script>
  var test = document.querySelector('div');
  test.onclick = function () {
    this.className = 'change';
  };
</script>
```



```
<script>
  var test = document.querySelector('div');
  test.onclick = function () {
    this.className = 'first change';
  };
</script>
```



6.5 操作元素

5

【案例】显示隐藏文本框内容

例6.5.5

案例分析: 本案例需要为一个文本框添加提示文本。当单击文本框时，里面的默认提示文字会隐藏，鼠标指针离开文本框，里面的文字会显示出来。



6.5 操作元素

5

【案例】显示隐藏文本框内容

实现步骤如下：

- ❑ 为元素绑定获取文本框焦点事件`onfocus`和失去焦点事件`onblur`
- ❑ 如果获取焦点时，需要判断表单里面的内容是否为默认文字；如果是默认文字，就清空表单内容
- ❑ 如果失去焦点，需要判断表单内容是否为空；如果为空，则表单里边的内容改为默认文字



6.5 操作元素

5

【案例】显示隐藏文本框内容

代码实现：步骤如下。

① 编写HTML结构，完成页面布局

示例代码

```
<body>  
  <input type="text" value="手机" style="color:#999">  
</body>
```



6.5 操作元素

5

【案例】显示隐藏文本框内容

② 编写实现获取焦点时效果的JavaScript代码

示例代码

```
<script>
var text = document.querySelector('input'); // 获取元素
text.onfocus = function () { // 注册 获得焦点事件 onfocus
    if (this.value === '手机') {
        this.value = '';
    }
    this.style.color = '#333';
};
```



6.5 操作元素

5

【案例】显示隐藏文本框内容

③ 编写实现失去焦点时效果的JavaScript代码

示例代码

```
text.onblur = function () { // 注册 失去焦点事件 onblur
    if (this.value === '') {
        this.value = '手机';
    }
    // 失去焦点需要把文本框里面的文字颜色变浅色
    this.style.color = '#999';
};
```

本章总结

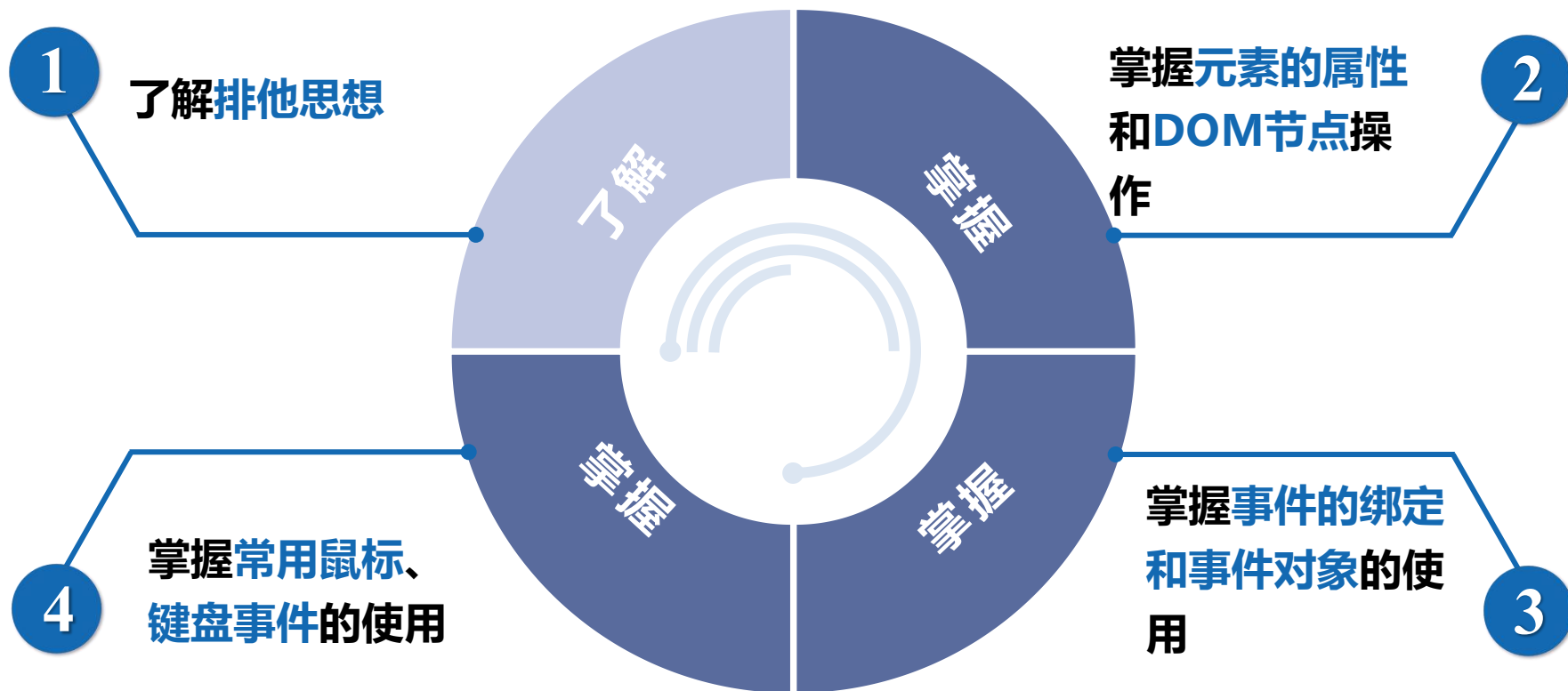


本章主要讲解了Web API的基本概念，如何利用DOM方式在JavaScript中获取元素，事件的基本概念，如何通过鼠标单击事件操作元素，如何对元素的内容、属性、样式进行操作，以及如何使用表单元素的获取焦点和失去焦点事件。通过本章的学习，读者应能够熟练地运用DOM完成元素的获取及操作。



第7章 DOM (下)

学习目标



目录



7.1

排他操作

👉 [点击查看本节相关知识点](#)

7.2

属性操作

👉 [点击查看本节相关知识点](#)

7.3

自定义属性

👉 [点击查看本节相关知识点](#)

7.4

节点基础

👉 [点击查看本节相关知识点](#)

目录



节点操作

☞ [点击查看本节相关知识点](#)



事件进阶

☞ [点击查看本节相关知识点](#)



事件对象

☞ [点击查看本节相关知识点](#)



鼠标事件

☞ [点击查看本节相关知识点](#)

目录



7.9

键盘事件

👉 [点击查看本节相关知识点](#)



7.1 排他操作

1

排他思想简介

排他思想，简单理解就是排除掉其他的（包括自己），然后再给自己设置想要实现的效果。总而言之，排他思想的实现步骤就是**所有元素全部清除与设置当前元素**。



7.1 排他操作

1

排他思想简介

例7.1.1

案例演示: 在同一组元素中，想要某一个元素实现某种样式，使用循环的排他思想算法来实现。

① 编写HTML结构代码

示例代码

```
<body>  
  <button>按钮1</button>  
  <button>按钮2</button>  
  <button>按钮3</button>  
  <button>按钮4</button>  
  <button>按钮5</button>  
</body>
```



7.1 排他操作

1

排他思想简介

② 编写JS代码，单击按钮，改变当前按钮背景色

示例代码

```
<script>
// 1. 获取所有按钮元素， btns得到的是类数组对象
var btns = document.getElementsByTagName('button');
for (var i = 0; i < btns.length; i++) { // 里面的每一个元素btns[i]
  btns[i].onclick = function () {
    for (var i = 0; i < btns.length; i++) { // (1) 先把所有的按钮背景颜色去掉
      btns[i].style.backgroundColor = '';
    }
    this.style.backgroundColor = 'pink'; // (2) 然后设置当前的元素背景颜色
  }
}
```



7.1 排他操作

1

排他思想简介

效果图如下所示：



排他思想



7.1 排他操作

2

【案例】鼠标指针经过时背景变色

例7.1.2

案例分析：当表格中的单元格比较多时，可以在用户鼠标指针经过时把当前行添加背景色，使表格内容显得清晰和一目了然，容易阅读。



7.1 排他操作

2

【案例】鼠标经过时背景变色

案例演示：接下来，使用鼠标指针经过事件`onmouseover`和鼠标指针离开事件`onmouseout`实现案例效果。CSS样式代码请参考本书源代码。

① 编写HTML结构代码



7.1 排他操作

2

【案例】鼠标经过时背景变色

示例代码

```
<table>
  <thead> <tr>
    <th>代码</th> <th>名称</th> <th>最新公布净值</th>
    <th>累计净值</th> <th>前单位净值</th> <th>净值增长率</th>
  </tr> </thead>
  <tbody> <tr>
    <td>0035**</td> <td>3个月定期开放债券</td> <td>1.075</td>
    <td>1.079</td> <td>1.074</td> <td>+0.047%</td>
  </tr>
  ... (此处省略多个tr)
</tbody>
</table>
```



7.1 排他操作

2

【案例】鼠标经过时背景变色

② 编写JS代码，鼠标指针经过时背景变色

示例代码

```
<script>
    var trs = document.querySelector('tbody').querySelectorAll('tr'); // 1. 获取元素
    for (var i = 0; i < trs.length; i++) { // 2. 利用循环绑定注册事件
        trs[i].onmouseover = function () { // 3. 鼠标经过事件 onmouseover
            this.className = 'bg';
        };
        trs[i].onmouseout = function () { // 4. 鼠标离开事件 onmouseout
            this.className = '';
        };
    }
</script>
```



7.1 排他操作

2

【案例】鼠标经过时背景变色

效果图如下所示：



代码	名称	最新公布净值	累计净值	前单位净值	净值增长率
0035**	3个月定期开放债券	1.075	1.079	1.074	+0.047%
0035**	3个月定期开放债券	1.075	1.079	1.074	+0.047%
0035**	3个月定期开放债券	1.075	1.079	1.074	+0.047%
0035**	3个月定期开放债券	1.075	1.079	1.074	+0.047%
0035**	3个月定期开放债券	1.075	1.079	1.074	+0.047%

背景变色效果



7.2 属性操作

1

获取属性值

在DOM对象中可以使用“**element.属性**”的方式来获取内置的属性值，但是DOM对象并不能直接使用点语法获取到自定义属性的值，那么如何获取自定义属性值呢？

在DOM中，可以使用[getAttribute\('属性'\)](#)方法来返回指定元素的属性值。接下来通过案例来演示如何获取属性值。



7.2 属性操作

1

获取属性值

例7.2.1

示例代码

```
<body>
  <div id="demo" index="1" class="nav"> </div>
  <script>
    var div = document.querySelector('div');
    console.log(div.id);           // 结果为: demo
    console.log(div.getAttribute('id')); // 结果为: demo
    console.log(div.getAttribute('index')); // 结果为: 1
  </script>
</body>
```



7.2 属性操作

2

设置属性值

在DOM对象中可以使用“`element.属性 = '值'`”的方式来设置内置的属性值，并且针对于自定义属性，提供了“`element.setAttribute('属性', '值')`”的方式进行获取。

值得一提的是，设置了自定义属性的标签，在浏览器中的HTML结构中可以看到该属性。接下来通过案例来演示如何设置属性值。



7.2 属性操作

2

设置属性值

例7.2.2

示例代码

```
<body>
  <div> </div>
  <script>
    var div = document.querySelector('div');
    div.id = 'test';           // 设置id值为test
    div.className = 'navs';    // 设置class类名为navs
    div.setAttribute('index', 2); // 设置属性名为index, 值为2
  </script>
</body>
```

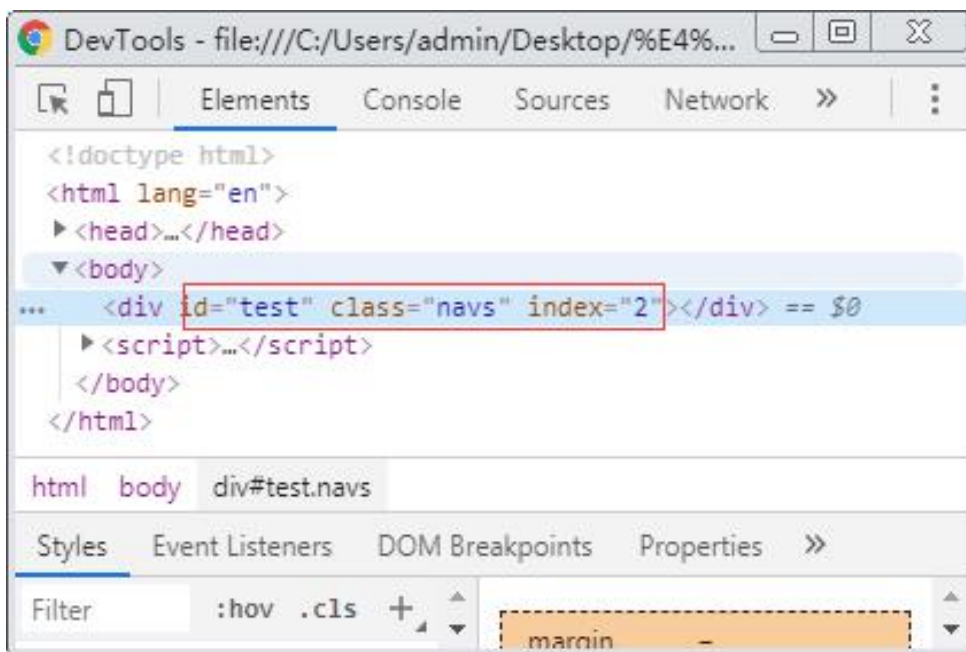


7.2 属性操作

2

设置属性值

效果图如下所示：



查看div元素的属性



7.2 属性操作

2

设置属性值

另外，也可以使用`setAttribute()`方式设置元素的类名。

示例代码

```
div.setAttribute('class', 'footer');
```

属性

类名

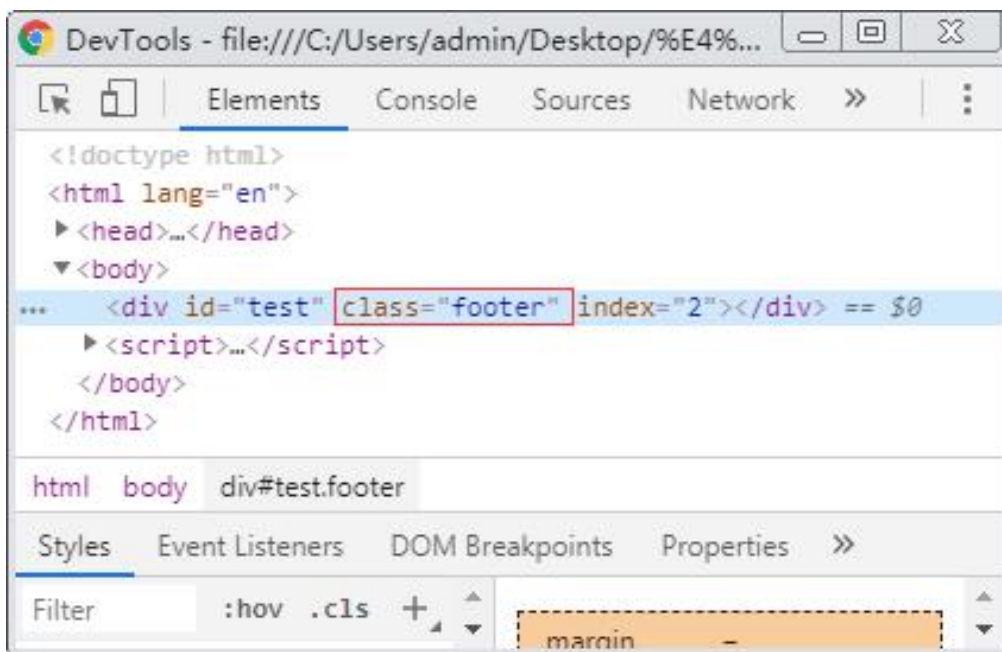


7.2 属性操作

2

设置属性值

效果图如下所示：



div元素的class属性



7.2 属性操作

3

移除属性值

例7.2.3

在DOM中使用 `element.removeAttribute('属性')` 的方式来移除元素属性。

接下来，通过案例演示如何移除属性值。

示例代码

```
<body>
  <div id="test" class="footer" index="2"> </div>
  <script>
    var div = document.querySelector('div');
    div.removeAttribute('id');    // 移除div元素的id属性
    div.removeAttribute('class'); // 移除div元素的class属性
    div.removeAttribute('index'); // 移除div元素的index属性
  </script>
</body>
```

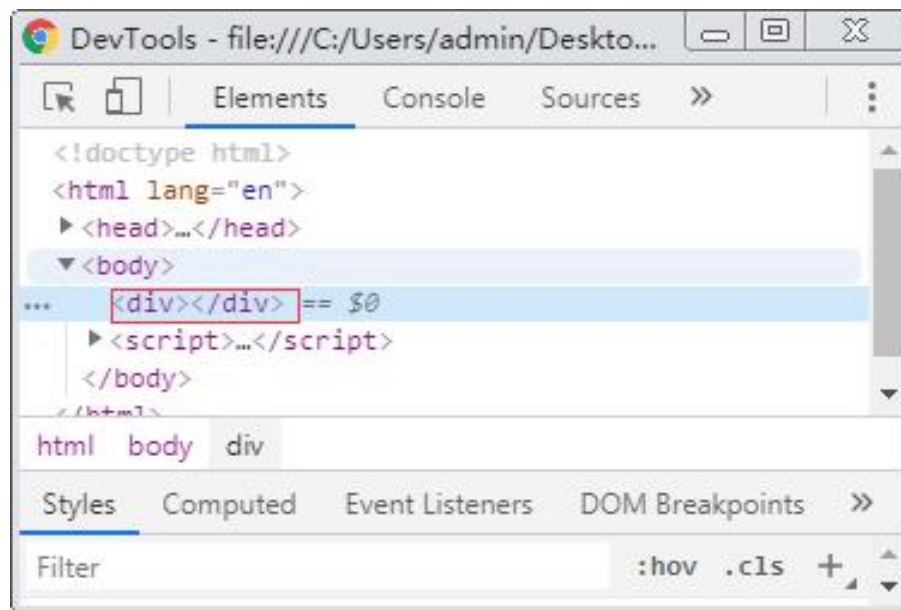


7.2 属性操作

3

移除属性值

效果图如下所示：



查看div元素



7.2 属性操作

4

【案例】Tab栏切换

例7.2.4

案例要求: 标签栏在网站中的使用非常普遍，它的优势在于可以在有限的空间内展示多块的内容，用户可以通过标签在多个内容块之间进行切换。

在本案例中要求使用自定义属性相关知识实现**Tab**栏切换效果，具体的**CSS**样式请参考本书源码。



7.2 属性操作

4

【案例】Tab栏切换

① 编写HTML结构代码

示例代码

```
<div class="tab">
  <div class="tab_list"></div> // 标签部分
  <div class="tab_con">        // 标签内容部分
    <div class="item" style="display: block;">商品介绍模块内容</div>
    <div class="item">规格与包装模块内容</div>
    <div class="item">售后保障模块内容</div>
    <div class="item">商品评价（50000）模块内容</div>
    <div class="item">手机社区模块内容</div>
  </div>
</div>
```

7.2 属性操作

4

【案例】Tab栏切换

效果图如下所示：



标签栏默认效果



7.2 属性操作

4

【案例】Tab栏切换

② 编写JS代码，获取标签部分和内容部分的对象元素

示例代码

```
<script>  
    // 获取标签部分的所有元素对象  
    var tab_list = document.querySelector('.tab_list');  
    var lis = tab_list.querySelectorAll('li');  
    // 获取内容部分的所有内容对象  
    var items = document.querySelectorAll('.item');  
</script>
```



7.2 属性操作

4

【案例】Tab栏切换

③ 编写JS代码，for循环绑定单击事件

示例代码

```
for (var i = 0; i < lis.length; i++) { // for循环绑定单击事件
    lis[i].setAttribute('index', i); // 开始给5个小li设置索引号
    lis[i].onclick = function () { // 清除所有li的class类，给自己设置current类
        for (var i = 0; i < lis.length; i++) { lis[i].className = ''; }
        this.className = 'current';
        // 2. 下面的显示内容模块
        var index = this.getAttribute('index');
        for (var i = 0; i < items.length; i++) { items[i].style.display = 'none'; }
        items[index].style.display = 'block';
    }
};
```

排他思想

7.2 属性操作

4

【案例】Tab栏切换

单击“售后保障”，效果图如下所示：



标签栏切换效果



7.3 自定义属性

1

设置属性值

一般的自定义属性可以通过`getAttribute('属性')`方法来获取，但是有些自定义属性很容易引起歧义，不容易判断是元素的自带属性还是自定义属性。因此，HTML5新增了自定义属性的规范，在HTML5中规定通过“[data-属性名](#)”的方式设置自定义属性。



7.3 自定义属性

1

设置属性值

元素的“data-*”自定义属性有两种设置方式，分别如下：

① 在HTML中设置自定义属性

示例代码

```
// 在div元素上设置data-index属性
```

```
<div data-index="2"> </div>
```

data-表示前缀，
index是自己定义的属性名



7.3 自定义属性

1

设置属性值

② 在JavaScript中设置自定义属性

在JavaScript代码中，可以通过[setAttribute\('属性', 值\)](#)或者“[元素对象.dataset.属性名='值'](#)”两种方式设置自定义属性。需要注意的是，通过后者的方式只能设置以“data-”开头的自定义属性。



7.3 自定义属性

1

设置属性值

示例代码

```
<body>
  <div></div>
  <script>
    var div = document.querySelector('div');
    div.dataset.index = '2';
    div.setAttribute('data-name', 'andy');
  </script>
</body>
```

浏览器预览结果

```
<div data-index="2" data-name="andy"></div>
```



7.3 自定义属性

2

获取属性值

在DOM操作中，提供了两种获取属性值的方式，第1种是通过[`getAttribute\(\)`](#)方式，该方式可以获取内置属性或者自定义属性；第2种是使用HTML5新增的“[`element.dataset.属性`](#)”或者“[`element.dataset\['属性'\]`](#)”方式（有兼容性问题）。接下来，通过案例形式演示如何获取属性值。



7.3 自定义属性

2

获取属性值

示例代码

```
<div getTime="20" data-index="2" data-list-name="andy"> </div>
<script>
  var div = document.querySelector('div');
  console.log(div.getAttribute('data-index'));    // 结果为: 2
  console.log(div.getAttribute('data-list-name')); // 结果为: andy
  // HTML5新增的获取自定义属性的方法, 只能获取 "data-" 开头的属性
  console.log(div.dataset); // DOMStringMap {index:"2",listName:"andy"}
  console.log(div.dataset.index);                // 结果为: 2
  console.log(div.dataset['index']);              // 结果为: 2
  console.log(div.dataset.listName);             // 结果为: andy
  console.log(div.dataset['listName']);          // 结果为: andy
</script>
```



7.3 自定义属性

2

获取属性值

注意

`dataset`是一个集合，里面存放了所有以`data`开头的自定义属性，如果自定义属性里面包含有多个连字符（-）时，获取的时候采取驼峰命名法。



7.4 节点基础

1

什么是节点

HTML文档可以看作是一个节点树，网页中的所有内容都是[节点](#)，一般来说，节点至少拥有`nodeType`（节点类型）、`nodeName`（节点名称）和`nodeValue`（节点值）这3个基本属性。下面列举常见的节点类型：

- ❑ 元素节点，`nodeType`为1
- ❑ 属性节点，`nodeType`为2
- ❑ 文本节点，`nodeType`为3，文本节点包含文字、空格、换行等



7.4 节点基础

2 节点层级

一个HTML文件可以看作是所有元素组成的一个节点树，各元素节点之间有级别的划分。

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>测试</title>
  </head>
  <body>
    <a href="#">链接</a>
    <p>段落...</p>
  </body>
</html>
```

DOM根据HTML中各节点的不同作用，可将其分别划分为标签节点（元素节点）、文本节点和属性节点。节点之间的层级关系如下：

- ❑ 根节点：<html>标签是整个文档的根节点，有且仅有一个
- ❑ 父节点：指的是某一个节点的上级节点
- ❑ 子节点：指的是某一个节点的下级节点
- ❑ 兄弟节点：两个节点同属于一个父节点

7.4 节点基础



2

节点层级

① 获取父级节点

例7.4.2-2

在JavaScript中，可以使用parentNode属性来获得离当前元素的最近的一个父节点，如果找不到父节点就返回为 null，语法格式为：obj.parentNode，obj是一个DOM对象。



7.4 节点基础

2

节点层级

案例演示: 如何获取当前元素的父节点。

示例代码

```
<body>
  <div class="demo">
    <div class="box"> <span class="child">span元素</span> </div>
  </div>
  <script>
    var child = document.querySelector('.child'); // 获取类名为child的span元素
    console.log(child.parentNode); // 输出离child元素最近的父级节点(box)
  </script>
</body>
```

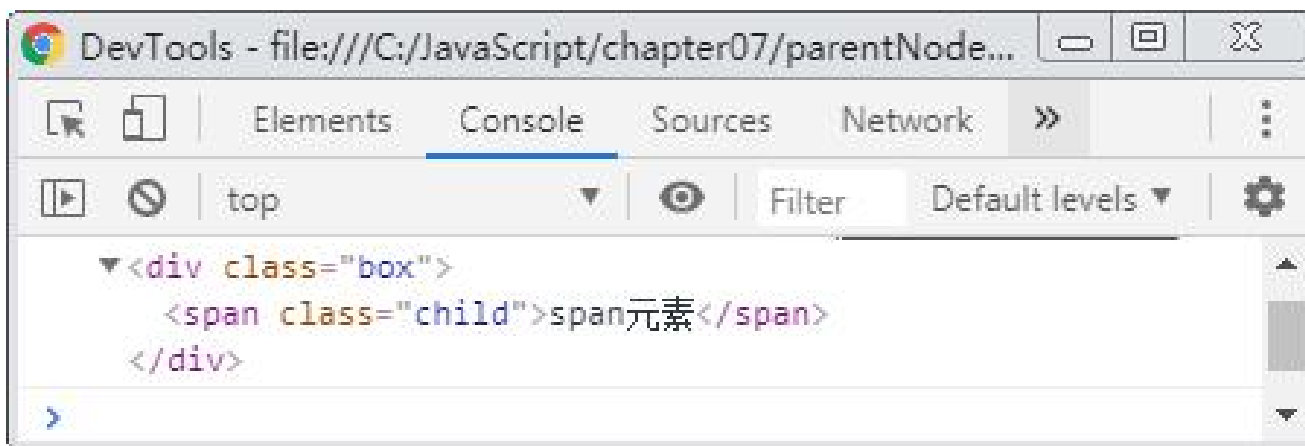


7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



父级节点



7.4 节点基础

2

节点层级

② 获取子级节点

在JavaScript中，可以使用`childNodes`属性或者`children`属性两种方式来获得当前元素的所有子节点的集合，接下来我们就分别介绍这两种方式的用法。

方式1: `childNodes`属性获得的是当前元素的所有子节点的集合，该集合为即时更新的集合。下面通过案例的形式演示如何获取当前元素子节点。



7.4 节点基础

2

节点层级

例7.4.2-3

案例演示: 如何使用childNodes属性获取当前元素的子节点。

示例代码

```
<ul><li>我是li</li><li>我是li</li><li>我是li</li><li>我是li</li></ul>
<script>
  var ul = document.querySelector('ul');
  var lis = ul.querySelectorAll('li');
  console.log(lis)
  console.log(ul.childNodes);
  console.log(ul.childNodes[0].nodeType);
  console.log(ul.childNodes[1].nodeType);
</script>
```



7.4 节点基础

2

节点层级

注意

`childNodes`属性返回的是`NodeList`对象的集合，返回值里面包含了元素节点、文本节点等其他类型的节点。

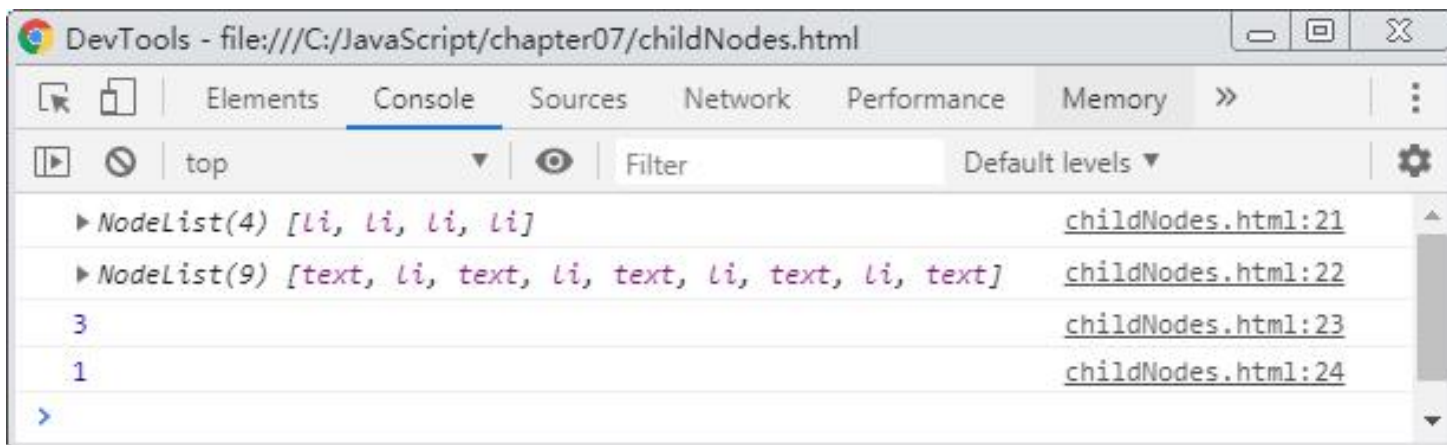


7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



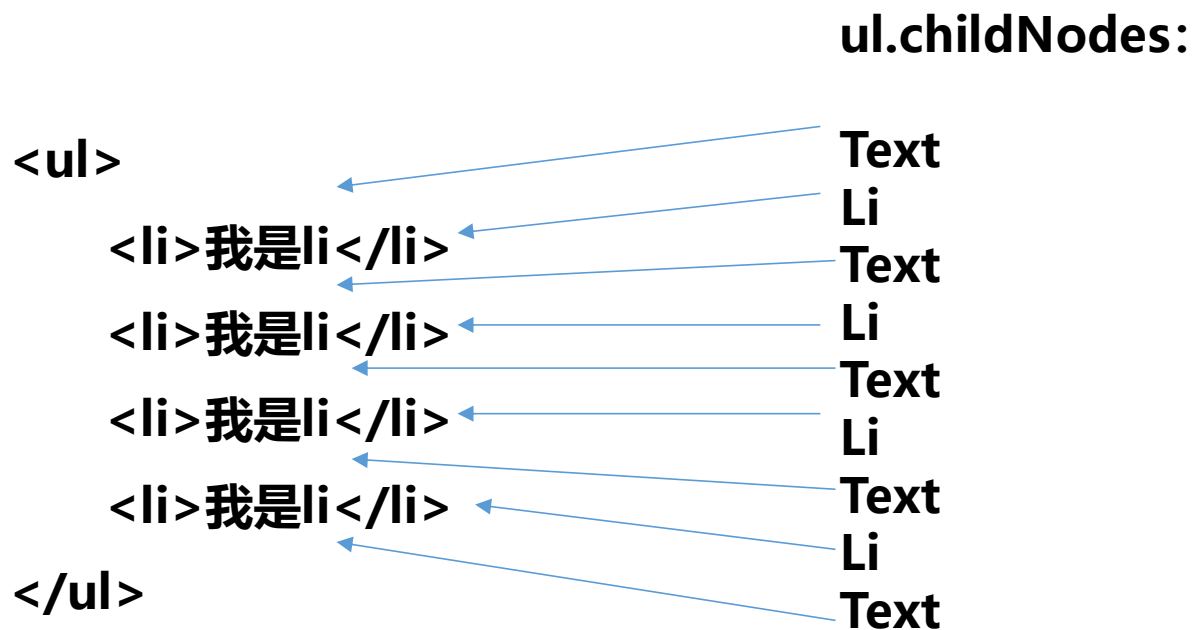
子级节点



7.4 节点基础

2

节点层级





7.4 节点基础

2

节点层级

方式2: `children`是一个可读的属性，返回所有子元素节点。`children`只返回子元素节点，其余节点不返回，目前各大浏览器都支持该属性，在实际开发中推荐使用`children`。

接下来，通过案例的形式演示如何获取当前元素子节点。



7.4 节点基础

2

节点层级

例7.4.2-4

案例演示: 如何使用children属性获取当前元素的子节点。

HTML代码

```
<ol>
  <li>我是li</li>
  <li>我是li</li>
  <li>我是li</li>
  <li>我是li</li>
</ol>
```

JS代码

```
<script>
  var ul = document.querySelector('ol');
  var lis = ul.querySelectorAll('li');
  console.log(ul.children); // 结果为: HTMLCollection(4) [li, li, li, li]
</script>
```

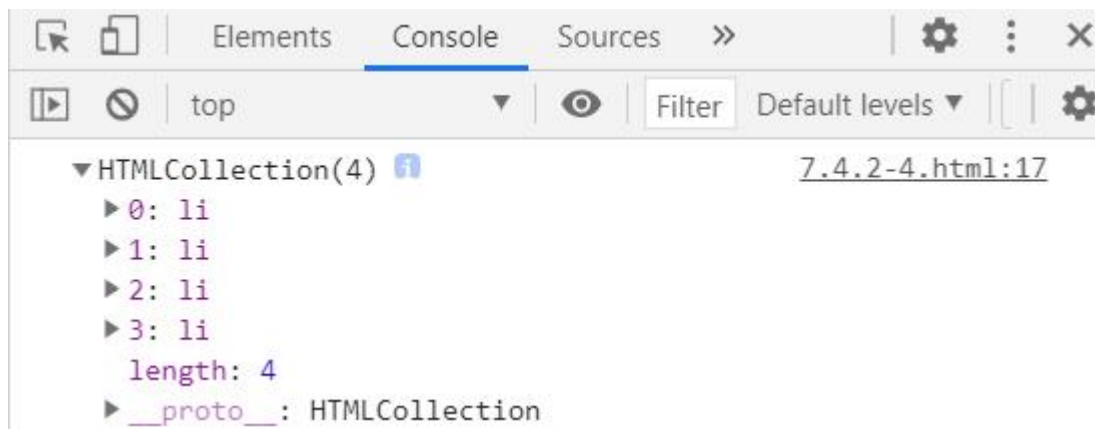


7.4 节点基础

2

节点层级

浏览器预览效果图如下所示：



子级节点



7.4 节点基础

2

节点层级

值得一提

`childNodes`属性与`children`属性虽然都可以获取某元素的子元素，但是两者之间有一定的区别。前者用于节点操作，返回值是`NodeList`对象集合，后者用于元素操作，返回的是`HTMLCollection`对象集合。



7.4 节点基础

2

节点层级

方式3: 使用firstChild属性和lastChild属性[获取子节点](#)，前者返回第一个子节点，后者返回的是最后一个子节点，如果找不到则返回null。需要注意的是它们的返回值包括文本节点和元素节点等。



7.4 节点基础

2

节点层级

方式4: 使用firstElementChild属性和lastElementChild属性获取[子元素节点](#)，前者返回第一个子元素节点，后者返回最后一个子元素节点，如果找不到则返回null。需要注意的是，**这两个属性有兼容性问题**，IE9以上才支持。



7.4 节点基础

2

节点层级

注意

实际开发中，`firstChild` 和 `lastChild` 包含其他节点，操作不方便，而 `firstElementChild` 和 `lastElementChild` 又有兼容性问题，为了解决兼容性问题，在实际开发中通常使用 “`obj.children[索引]`” 的方式来获取子元素节点。

示例代码

```
obj.children[0]
```

```
// 获取第一个子元素节点
```

```
obj.children[obj.children.length - 1]
```

```
// 获取最后一个子元素节点
```



7.4 节点基础

2

节点层级

③ 获取兄弟节点

在JavaScript中，可以使用[nextSibling](#)属性获得下一个兄弟节点，或者使用[previousSibling](#)属性获得上一个兄弟节点，它们的返回值包含元素节点或者文本节点等。如果找不到，就返回null。



7.4 节点基础

2

节点层级

可以使用`nextElementSibling`返回当前元素的下一个兄弟元素节点，`previousElementSibling`属性返回当前元素的上一个兄弟元素节点，如果找不到则返回`null`。需要注意的是，这两个属性有兼容性问题，IE9以上才支持。



7.4 节点基础

2

节点层级

实际开发中，`nextSibling`和`previousSibling`属性返回值都包含其他节点，操作不方便，而`nextElementSibling`和`previousElementSibling`又有兼容性问题。

为了解决兼容性问题，在实际开发中通常使用封装函数来处理兼容性。



7.4 节点基础

2

节点层级

示例代码

```
function getNextElementSibling(element) {  
  var el = element;  
  while (el = el.nextSibling) {  
    if (el.nodeType === 1) {  
      return el;  
    }  
  }  
  return null;  
}
```



7.4 节点基础

3

【案例】下拉菜单

例7.4.3

案例需求: 鼠标指针经过菜单时，显示当前下拉框中的内容同时隐藏其他下拉菜单内容

① 编写HTML结构代码

示例代码

```
<body>
  <ul class="nav">
    <li>
      <a href="#">微博</a>
      <ul></ul>
    </li>
    ... (此处省略3个li)
  </ul>
</body>
```

a为标签名，ul为下拉内容



7.4 节点基础

3

【案例】下拉菜单

② 鼠标经过，展示当前下拉列表内容

示例代码

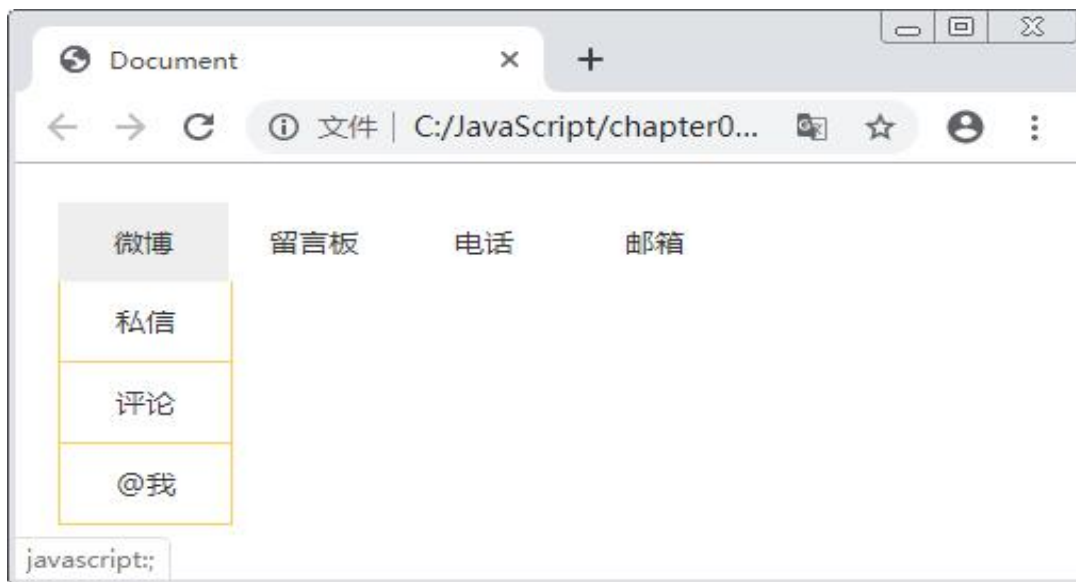
```
<script>
var nav = document.querySelector('.nav'); // 1. 获取元素
var lis = nav.children;
for (var i = 0; i < lis.length; i++)
lis[i].onmouseover = function () { // 2. 注册鼠标指针经过和鼠标指针离开事件
    this.children[1].style.display = 'block'; // ul为li的第2个子元素
};
lis[i].onmouseout = function () { this.children[1].style.display = 'none'; };
}
</script>
```

7.4 节点基础

3

【案例】下拉菜单

浏览器预览效果图如下所示：



下拉菜单效果



7.5 节点操作

1

创建节点

在DOM中，使用[document.createElement\('tagName'\)](#)方法创建由tagName指定的HTML元素，也称为动态创建元素节点。

动态创建元素节点的3种常见方式如下：

- ❑ document.write()创建元素，如果页面文档流加载完毕，再调用会导致页面重绘
- ❑ element.innerHTML将内容写入某个DOM节点，不会导致页面全部重绘
- ❑ document.createElement()创建多个元素效率稍微低一点，但是结构更加清晰



7.5 节点操作

2

添加和删除节点

DOM中，提供了[`node.appendChild\(\)`](#)和[`node.insertBefore\(\)`](#)方法用于添加节点，[`node.removeChild\(child\)`](#)用于删除节点。

下面讲解这3种方法的使用。

- ❑ `appendChild()`方法，将一个节点添加到指定父节点的子节点列表末尾
- ❑ `insertBefore(child, 指定元素)`方法，将一个节点添加到父节点的指定子节点前面
- ❑ `removeChild(child)`用于删除节点，该方法从DOM中删除一个子节点，返回删除的节点



7.5 节点操作

3

【案例】简易留言板

例7.5.3

案例分析：利用节点的创建、添加和删除相关知识完成一个简易的留言板功能。在页面中实现单击“发布”按钮动态创建一个li元素，添加到ul里面。案例实现步骤如下：



7.5 节点操作

3

【案例】简易留言板

① 编写HTML结构，完成页面布局

示例代码

```
<body>  
  <textarea name="" id=""></textarea> // 用户输入留言  
  <button>发布</button>  
  <ul></ul> // 用于展示留言模块  
</body>
```



7.5 节点操作

3

【案例】简易留言板

② 添加留言功能: 获取元素

示例代码

```
<script>  
  // 1. 获取元素  
  var btn = document.querySelector('button');  
  var text = document.querySelector('textarea');  
  var ul = document.querySelector('ul');  
</script>
```



7.5 节点操作

3

【案例】简易留言板

② 添加留言功能: 注册事件, 添加留言

示例代码

```
btn.onclick = function () {  
    if (text.value == '') {  
        alert('您没有输入内容');  
        return false;  
    } else { // (1) 创建元素  
        var li = document.createElement('li');  
        li.innerHTML = text.value;  
        ul.insertBefore(li, ul.children[0]); // (2) 添加元素  
        // ... (3) 此处编写删除留言功能  
    }  
};
```

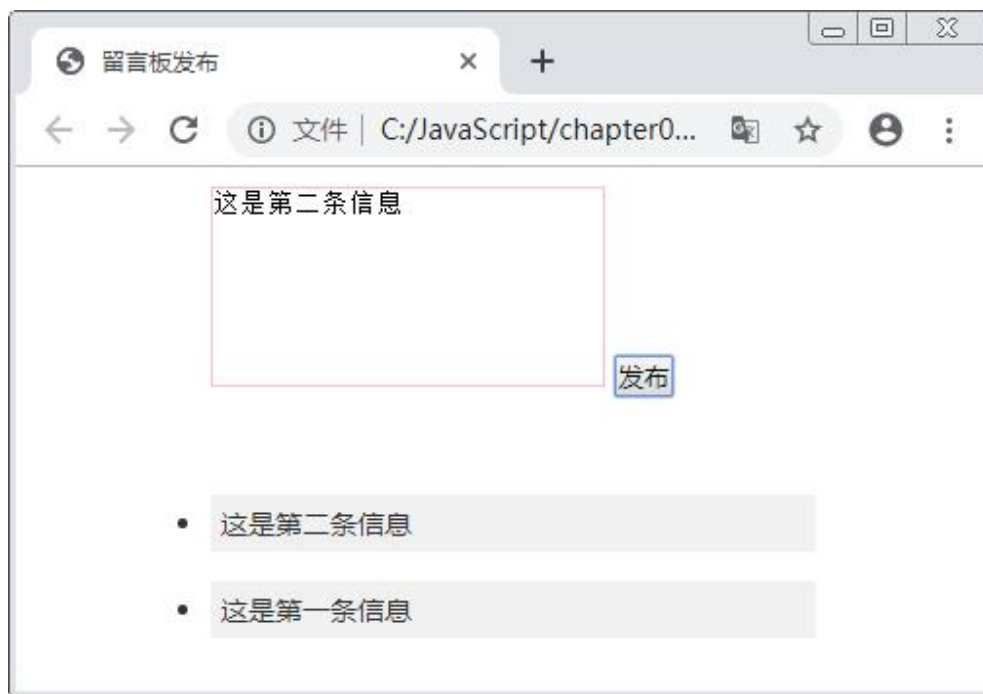


7.5 节点操作

3

【案例】简易留言板

浏览器预览效果图如下所示：



添加留言



7.5 节点操作

3

【案例】简易留言板

- ③ 删除留言功能: 首先在添加li时, 给li中增加一个a链接, 给所有的a链接注册单击事件, 找到a的父节点li, 进行删除即可。

修改第 (2) 步中代码, 找到如下所示代码

```
li.innerHTML = text.value;
```

修改为

```
li.innerHTML = text.value + '<a href="javascript:;">删除</a>';
```



7.5 节点操作

3

【案例】简易留言板

③ 删除留言功能: 注册事件, 删除留言

示例代码

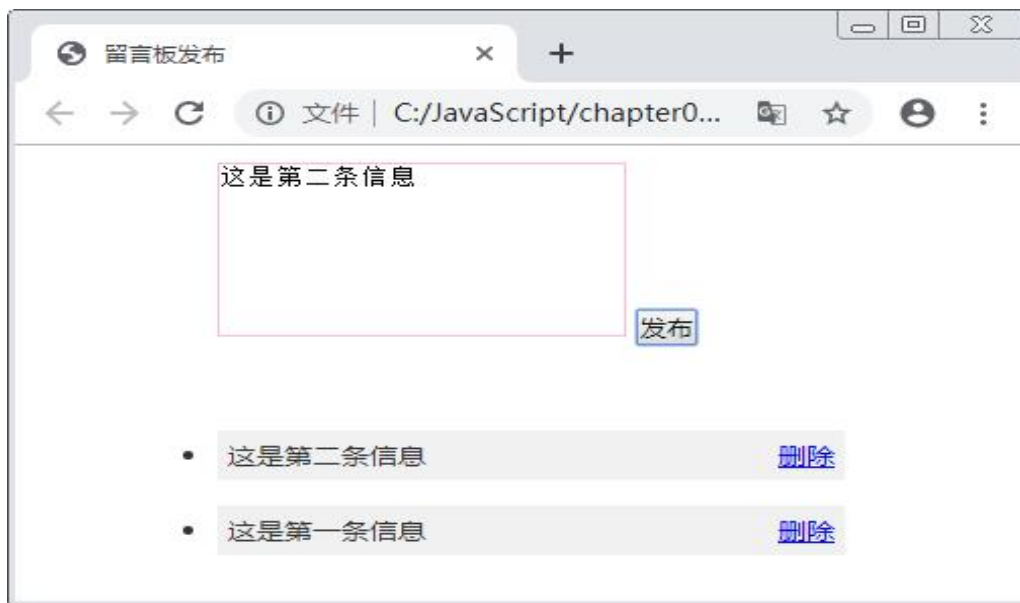
```
// 在(3)处编写以下代码
var as = document.querySelectorAll('a');
for (var i = 0; i < as.length; i++) {
  as[i].onclick = function () {
    ul.removeChild(this.parentNode);
  };
}
```

7.5 节点操作

3

【案例】简易留言板

浏览器预览效果图如下所示：



删除留言



7.5 节点操作

4

复制节点

在DOM中，提供了[node.cloneNode\(\)](#)方法，返回调用该方法的节点的一个副本，也称为克隆节点或者拷贝节点。语法为“**需要被复制的节点.cloneChild(true/false)**”。如果参数为空或**false**，则是**浅拷贝**，即**只复制节点本身，不复制里面的子节点**；如果括号参数为**true**，则是**深拷贝**，即会**复制节点本身及里面所有的子节点**。



7.5 节点操作

4

复制节点

例7.5.4

示例代码

```
<body>
  <ul id="myList"> <li>苹果</li> <li>橙子</li> <li>橘子</li> </ul>
  <ul id="op"> </ul>
  <button onclick="myFunction()">点我</button>
  <script>
    function myFunction() {
      var item = document.getElementById('myList').firstChild;
      var cloneItem = item.cloneNode(true);
      document.getElementById('op').appendChild(cloneItem);
    }
  </script>
</body>
```

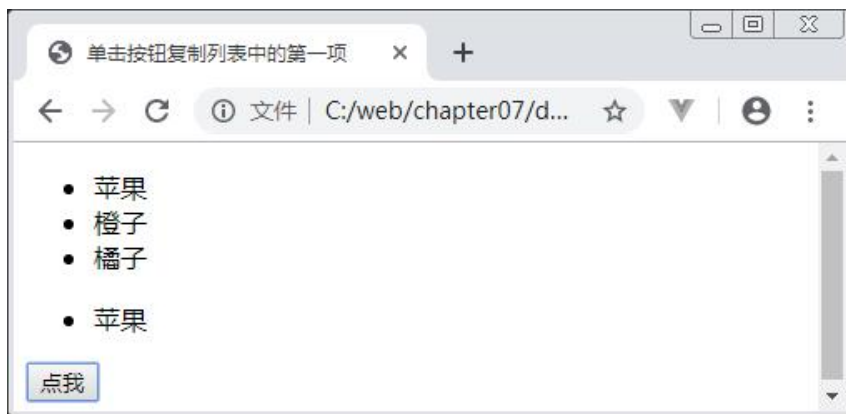


7.5 节点操作

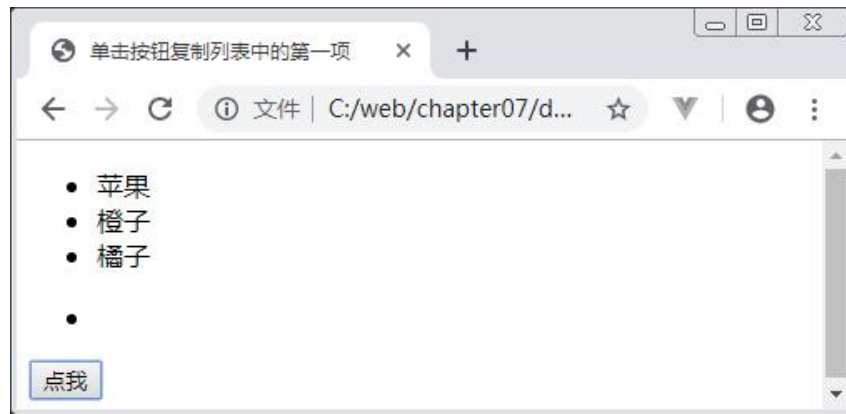
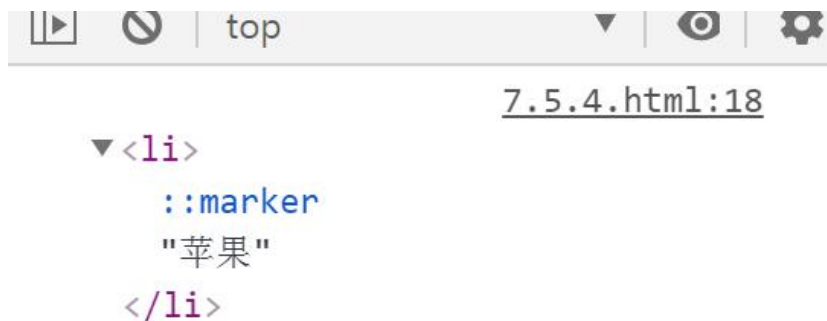
4

复制节点

当单击“点我”按钮的时候，浏览器预览效果图如下所示：



cloneNode(true)



cloneNode(false)

